



INDIA.RUNS

Build what next India runs on

Team Name : Harshada Rameshwar Wagh

Team Leader Name : Harshada Rameshwar Wagh

Problem Statement : Intelligent Candidate Discovery

Solution Overview

What is your proposed solution?

- ❑ **We created a candidate ranking system that reads the job description and compares it with candidate profiles. Instead of using only keywords, we combined career history, skills, keyword matching and behavioral signals to calculate a final score. Finally, the system ranks the top 100 candidates and generates an explainable reasoning for each shortlisted candidate.**
- What differentiates your approach from traditional candidate matching systems?
- ❑ **Traditional systems mostly depend on keyword matching. Our approach considers multiple factors such as career history, skills, recruiter response rate, GitHub activity, notice period and open-to-work status. These signals are combined using a weighted scoring method, and every shortlisted candidate includes a reasoning so that the ranking is easy to understand.**

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?
 - ❑ **Experience in AI/ML**
 - ❑ **Search or Recommendation Systems**
 - ❑ **Strong career history relevant to the role**
 - ❑ **Relevant technical skills Good project experience**
 - ❑ **Active and job-ready candidate profile**
- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?
 - ❑ **Career history**
 - ❑ **Skills**
 - ❑ **Job description keyword match**
 - ❑ **Recruiter response rate**
 - ❑ **GitHub activity Notice period**
 - ❑ **Open-to-work status**
 - ❑ **Notice Period**

Ranking Methodology

- How does your system retrieve, score, and rank candidates?
- ❑ **Our system first extracts the important requirements from the Job Description. It then evaluates every candidate using career history, skills, keyword relevance and behavioral signals. Individual scores are calculated for each factor and combined into a final weighted score. Finally, candidates are ranked in descending order and the Top 100 candidates are shortlisted.**
- What models, algorithms, or heuristics are used?
- ❑ **Rule-based keyword matching Career history scoring Skill matching Behavioral signal scoring Weighted hybrid ranking Rule-based reasoning generation**
- How are multiple candidate signals combined into a final ranking?
- ❑ **We normalize individual scores and combine them using weighted scoring. Career history receives the highest importance, followed by keyword relevance, behavioral signals and skills. The final score is used to rank candidates, and a recruiter-style reasoning is generated for each shortlisted profile**

Explainability & Data Validation

- How are ranking decisions explained?
 - ❑ **Every shortlisted candidate includes a short reasoning based on career history, skills, behavioral signals and job relevance. The reasoning highlights why the candidate matches the role.**
- How do you prevent hallucinations or unsupported justifications?
 - ❑ **We only use information available in the provided dataset. The reasoning is generated from actual candidate data such as career history, skills and behavioral signals. No external information or assumptions are added.**
- How does your solution handle inconsistent, low-quality, or suspicious profiles?
 - ❑ **Profiles with weak career history, low skill relevance, poor behavioral signals or suspicious activity receive lower scores. We also applied a honeypot penalty to reduce the ranking of suspicious candidates.**

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?

Job Description



Extract Key Requirements



Load Candidate Profiles



Career History Scoring



Skill Matching



Keyword Matching



Behavior Signal Scoring



Weighted Final Score



Apply Honeypot Penalty



Generate Recruiter-Style Reasoning



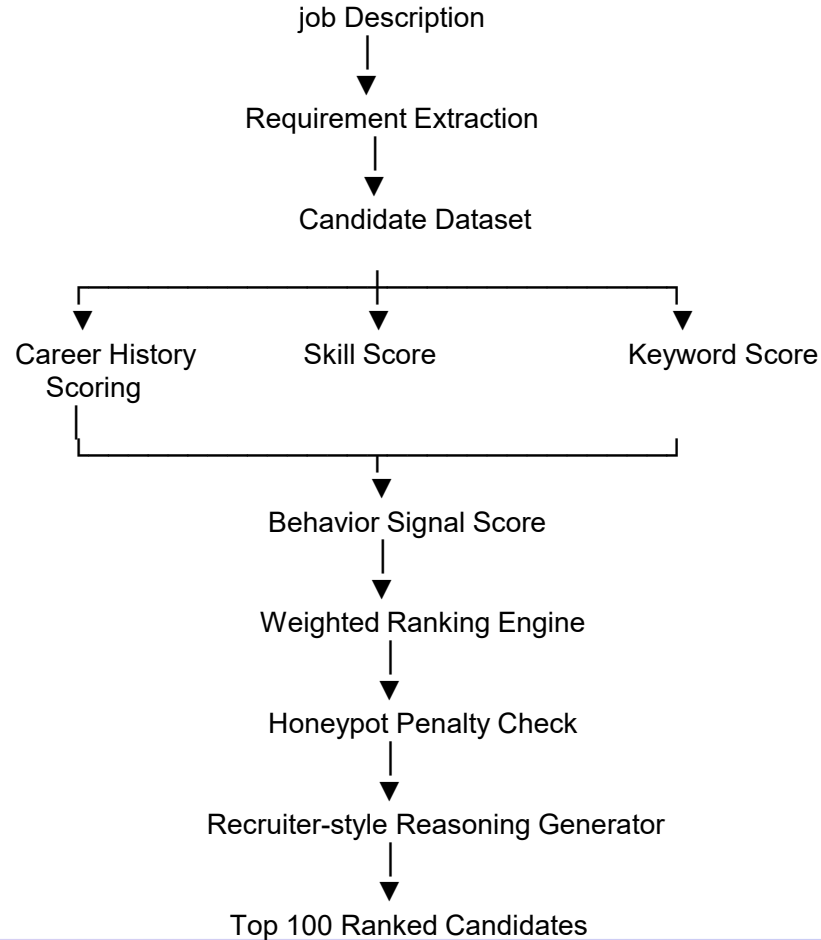
Rank Candidates



Top 100 Recommended Candidates

The system processes the job description, evaluates every candidate using multiple scoring components, combines them into a weighted final score, applies quality checks, generates explainable reasoning, and finally recommends the Top 100 candidates.

System Architecture



Results & Performance

- What results or insights demonstrate ranking quality?
 - ❑ Successfully ranked the Top 100 candidates
 - ❑ .Ranking is based on multiple candidate signals instead of only keywords.
 - ❑ Every shortlisted candidate includes an explainable recruiter-style reasoning.
 - ❑ Top-ranked candidates showed strong alignment with the job requirements.
 - ❑ Suspicious or low-quality profiles received lower priority through the honeypot penalty.
- How does your solution meet the challenge's runtime and compute constraints?
 - ❑ Implemented using Python and Pandas for efficient processing.
 - ❑ Lightweight rule-based scoring without expensive model inference.
 - ❑ Processes the dataset quickly on standard hardware.
 - ❑ Memory-efficient approach suitable for large candidate datasets.

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?

Technology / Tool	Purpose
Python	Core programming language used to build the candidate ranking pipeline.
Pandas	Data loading, preprocessing, feature engineering and ranking.
NumPy	Mathematical calculations and score normalization.
OpenPyXL	Exporting the final ranked candidates into an Excel file (submission.xlsx).
Jupyter Notebook	Development, testing and implementation of the complete workflow.
GitHub	Version control and project submission.

Submission Assets

GitHub Repository = <https://github.com/harsha123-code/redrob-candidate-ranking>

Files Submitted

- candidate_ranking.ipynb
- submission.xlsx
- README.md
- requirements.txt
- Intelligent_Candidate_Discovery.pdf



INDIA.RUNS

Build what next India runs on

THANK YOU